

Predictive Sensors and Heating Behavior via ML

Let's create a real-world predictive ML example based on a **limited dataset obtained from sensors of an industrial machine**. Due to data availability constraints, the dataset will be small (50 rows) and reflects realistic sensor behavior. The example will be sensor-based, predicting a target variable such as a machine temperature anomaly from multiple sensor readings. The project will include a clear problem statement, the dataset description, and complete Python code using standard machine-learning libraries.

```
x@x-virtual-machine: ~/Documents/dataset/ML_sensor
GNU nano 6.2 predictive_model.py
predictive_model.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# -----
# Load the generated dataset
# -----
df = pd.read_csv('/home/x/sensor_data.csv')
print("Dataset Loaded:")
print(df.head())

# -----
# Prepare features and target
# -----
X = df[['vibration', 'pressure', 'humidity']]
y = df['overheat']

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Train Random Forest Classifier
# -----
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
x@x-virtual-machine: ~/Documents/dataset/ML_sensor
GNU nano 6.2 predictive_model.py
# -----
# Train Random Forest Classifier
# -----
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# -----
# Make predictions
# -----
y_pred = model.predict(X_test)

# -----
# Evaluate the model
# -----
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# -----
# Predict on a new sensor reading
# -----
new_sensor = pd.DataFrame({
    'vibration': [8.5],
    'pressure': [85],
    'humidity': [60]
})
prediction = model.predict(new_sensor)
print("\nNew Sensor Prediction (1=Overheat, 0=Normal):", prediction[0])
```

```
x@x-virtual-machine:~/Documents/datasc/ML_sensor$ python3 predictive_model.py
Dataset Loaded:
  vibration  pressure  humidity  overheat
0   3.745401  97.566770  31.885751      1
1   9.507143  82.010626  68.184625      1
2   7.319939  95.159915  48.861359      1
3   5.986585  91.586188  60.514241      1
4   1.560186  67.831998  84.453988      0

Accuracy: 0.9

Confusion Matrix:
[[4 1]
 [0 5]]

Classification Report:
              precision    recall  f1-score   support

     0           1.00      0.80      0.89         5
     1           0.83      1.00      0.91         5

 accuracy          0.90      0.90      0.90         10
 macro avg          0.92      0.90      0.90         10
weighted avg          0.92      0.90      0.90         10

New Sensor Prediction (1=Overheat, 0=Normal): 1
```

Interpretation

✓ What just happened (high level)

- Dataset loaded correctly ✓
- Model trained on **80% of data** ✓
- Tested on **20% unseen data** ✓
- Predictions + evaluation successful ✓

Understanding the Output

Accuracy: 0.9

➡ The model correctly predicted **9 out of 10** test samples.

Confusion Matrix

[[4 1]

[0 5]]

Interpretation:

	Predicted 0	Predicted 1
Actual 0 (TN)	4	1 (FP)
Actual 1 (FN)	0	5 (TP)

- **True Negatives (4)** → Normal machines correctly predicted
- **False Positives (1)** → Normal predicted as overheat
- **False Negatives (0)** → ! None missed (very good for safety)
- **True Positives (5)** → Overheating correctly detected

💡 In real-world maintenance, **zero false negatives** is excellent.

Classification Report

For class 1 (Overheat):

- **Precision = 0.83** → When model says “overheat”, it’s right 83% of the time
- **Recall = 1.00** → It caught **all overheating cases**
- **F1-score = 0.91** → Balanced performance

This is exactly what you want in **predictive maintenance systems**.

New Sensor Prediction

New Sensor Prediction (1=Overheat, 0=Normal): 1

For:

- Vibration = 8.5
- Pressure = 85
- Humidity = 60

➡ Model predicts **OVERHEAT** — consistent with training logic.

🧠 One-line explanation you can say confidently

“The Random Forest model was trained on 80% of sensor data, evaluated on 20% unseen data, achieved 90% accuracy, and successfully predicts overheating conditions for new sensor readings.”

🚀 We now have a **COMPLETE ML PROJECT**

- ✓ Data generation
- ✓ File-based dataset
- ✓ Training + testing
- ✓ Evaluation metrics
- ✓ Real-world use case